Assignment 1

Full Stack Web Development — Questions & Answers

Question 1

Write a folder structure (as a tree) for a full stack e-commerce app using React (frontend), Express (backend), and MySQL (database). In 2 lines, justify why the `routes/` and `models/` folders should never import each other directly.

Answer:

```
my-ecommerce-app/
|-- client/                    # React Frontend
|   |-- public/
|   `-- src/
|       |-- components/
|       |-- pages/
|       `-- App.js
|-- server/                    # Express Backend
|   |-- config/                # DB Connection
|   |-- controllers/           # Business Logic
|   |-- middleware/            # Middlewares
|   |-- models/                # DB Schemas/Queries
|   |-- routes/                # Express Routes
|   `-- server.js
|-- .env
|-- .gitignore
`-- package.json
```

Justification:

Direct imports between `routes/` and `models/` break the separation of concerns in MVC architecture by tightly coupling routing logic with database operations. Using `controllers/` as an intermediary keeps the application modular, readable, and easier to test.

Question 2

This React component fetches data using a hardcoded localhost URL. Explain in 2 lines why this call will fail once deployed to production, and rewrite the fetch line using an environment variable.

```
function ProductList() {
  useEffect(() => {
    fetch("http://localhost:5000/api/products")
      .then(res => res.json())
      .then(setProducts);
  }, []);
  ...
}
```

Why it fails:

`localhost` refers specifically to the user's local machine, so production clients will attempt to fetch from their own device rather than your deployed server domain.

Rewritten Line:

```
fetch(`${process.env.REACT_APP_API_BASE_URL}/api/products`)
```

Question 3

Write a .env file and the corresponding dotenv-loading code for a Node.js/Express backend that needs PORT, DB_URL, and JWT_SECRET. In 1 line, explain why .env should never be committed to Git.

.env File:

```
PORT=5000
DB_URL=mysql://root:password@localhost:3306/ecommerce_db
JWT_SECRET=super_secret_jwt_key_12345
```

Express Setup Code:

```
require('dotenv').config();

const PORT = process.env.PORT || 5000;
const DB_URL = process.env.DB_URL;
const JWT_SECRET = process.env.JWT_SECRET;
```

Why it shouldn't be in Git:

It contains confidential credentials and secrets that expose your system to critical security vulnerabilities if pushed to version control.

Question 4

A React frontend calling an Express API gets a CORS error in the browser console on every request. Write the exact Express middleware/package code that fixes it, and explain in 1 line why the browser blocks the request in the first place.

Express Middleware Fix:

```
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors()); // Enables Cross-Origin Resource Sharing globally
```

Why Browser Blocks It:

Browsers enforce the Same-Origin Policy (SOP) to restrict web pages from making cross-domain API requests without explicit server permission.

Question 5

Design a REST API contract (as a table: Endpoint | Method | Request Body | Response) for a "Book Library" system with 4 endpoints covering create, read, update, and delete.

Answer:

Endpoint	Method	Request Body	Response
`/api/books`	GET	None	200 OK — Array of book objects
`/api/books`	POST	`{ "title": "String", "author": "String" }`	201 Created — Created book object
`/api/books/:id`	PUT	`{ "title": "String", "author": "String" }`	200 OK — Updated book object
`/api/books/:id`	DELETE	None	200 OK — { "message": "Book deleted" }

Question 6

This Express code creates a new database connection inside every route handler. Rewrite it so the connection is created once and reused across routes, and explain in 2 lines why the original version is a performance problem.

```
app.get('/products', (req, res) => {
  const db = mysql.createConnection({ host: 'localhost', user: 'root' });
  db.query('SELECT * FROM products', (err, rows) => res.json(rows));
```

```
});
```

Rewritten Code:

```
const express = require('express');
const mysql = require('mysql2');
const app = express();

// Create connection pool once
const db = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'ecommerce'
});

app.get('/products', (req, res) => {
  db.query('SELECT * FROM products', (err, rows) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(rows);
  });
});
```

Performance Problem:

Establishing a fresh database connection on every request causes huge performance overhead due to repeated TCP handshakes. Under high traffic, this rapidly exhausts system memory and connection limits.

Question 7

Write the "scripts" section of package.json for a full stack project where `npm run dev` should start both the backend (nodemon server.js) and the frontend (npm start inside a client folder) together, using the concurrently package.

Answer:

```
"scripts": {
  "start": "node server.js",
  "server": "nodemon server.js",
  "client": "npm start --prefix client",
  "dev": "concurrently \"npm run server\" \"npm run client\""
}
```

Question 8

A teammate's code hardcodes an API key directly inside a React component. Rewrite the code using an environment variable instead, and explain the security risk in 2 lines.

```
const response = await fetch(
  `https://api.weather.com/data?key=sk_live_9f8a7b6c5d4e`
);
```

Rewritten Code:

```
const response = await fetch(
  `https://api.weather.com/data?key=${process.env.REACT_APP_WEATHER_API_KEY}`
);
```

Security Risk:

Hardcoding API keys inside client-side code exposes them publicly in browser developer tools and compiled JS bundles. Malicious actors can extract the key to exhaust your quota or gain unauthorized access.

Question 9

Write pseudocode (as commented steps) tracing the full request-response cycle when a user submits a login form in a full stack app — from the button click to the database query to the UI update.

Answer:

```
// Step 1: Frontend - User fills out the login form and clicks "Submit".
// Step 2: Frontend - Event listener prevents default form behavior and
//         collects state inputs (username, password).
// Step 3: Frontend - Sends an HTTP POST request using fetch/axios to the
//         backend endpoint (/login) with a JSON payload.
// Step 4: Backend - Express server receives the incoming request and passes
//         it through body-parsing middleware.
// Step 5: Backend - Route controller extracts credentials and executes a
//         query to the database.
// Step 6: Database - Checks user credentials against stored records and
//         returns match status.
// Step 7: Backend - Validates response, generates auth payload/JWT token,
//         and returns an HTTP 200 response.
// Step 8: Frontend - Receives response, saves token to client storage, and
//         redirects user to update the UI.
```

Question 10

This Express server crashes on the very first POST request. Identify the missing line that causes `req.body` to be undefined, and provide the corrected code.

```
const express = require('express');
const app = express();
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  res.send(`Welcome ${username}`);
});
app.listen(3000);
```

Missing Line:

```
app.use(express.json());
```

Corrected Code:

```
const express = require('express');
const app = express();

// Parse incoming JSON payloads
app.use(express.json());

app.post('/login', (req, res) => {
  const { username, password } = req.body;
  res.send(`Welcome ${username}`);
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```